

**LAPORAN PRAKTIKUM
PRAKTIKUM PENGUJIAN PERANGKAT LUNAK**

PERTEMUAN 7

Selenium



Disusun Oleh:

Nama : Abdullah Afif Habiburrohman
NIM : 537611/SV/24441
Kelas : BB / B2
Dosen : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

**PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

A. Hasil dan Pembahasan

Pada praktikum kali ini, mahasiswa akan menguji aplikasi web menggunakan Selenium WebDriver. Selenium WebDriver adalah sebuah alat yang digunakan untuk mengotomatisasi pengujian aplikasi web dengan cara mensimulasikan interaksi pengguna pada browser. Dalam praktikum ini, mahasiswa akan membuat skrip pengujian untuk menguji fungsionalitas web `saucedemo.com`. Tujuannya adalah untuk menguji dua user yang berbeda, yaitu "standard_user" dan "locked_out_user", untuk memastikan bahwa aplikasi web berfungsi sesuai dengan yang diharapkan.

A.1 Persiapan

Mahasiswa membuat proyek Java pada IntelliJ IDEA, kemudian menambahkan dependensi Selenium pada file `pom.xml`. Selain itu, mahasiswa juga menambahkan dependensi JUnit untuk melakukan pengujian unit pada skrip yang dibuat. Berikut adalah konfigurasi `pom.xml` yang mahasiswa gunakan:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         ↪ http://maven.apache.org/xsd/maven-4.0.0.xsd">
6     ...
7     <dependencies>
8         <!-- Source: https://mvnrepository.com/artifact/org.
9             ↪ seleniumhq.selenium/selenium-java -->
10        <dependency>
11            <groupId>org.seleniumhq.selenium</groupId>
12            <artifactId>selenium-java</artifactId>
13            <version>4.39.0</version>
14            <scope>compile</scope>
15        </dependency>
16        <!-- Source: https://mvnrepository.com/artifact/org.
17            ↪ junit.jupiter/junit-jupiter -->
18        <dependency>
19            <groupId>org.junit.jupiter</groupId>
20            <artifactId>junit-jupiter</artifactId>
21            <version>5.13.4</version>
22            <scope>test</scope>
23        </dependency>
24    </dependencies>
25</project>
```

Link-link yang digunakan untuk dependensi dapat dilihat pada komentar dalam kode, semuanya dari maven repository.

Kemudian, buat kelas pengujian pada `src/test/java/org/example` dengan nama `MainTest`. Mahasiswa mengimpor dependensi yang diperlukan dan menyiapkan `@AfterEach` untuk menutup browser setelah setiap pengujian. Berikut adalah struktur awal kelas `MainTest`:

```

1 package org.example;
2
3 import org.junit.jupiter.api.AfterEach;
4 import org.junit.jupiter.api.Assertions;
5 import org.junit.jupiter.api.Test;
6 import org.openqa.selenium.By;
7 import org.openqa.selenium.WebDriver;
8 import org.openqa.selenium.WebElement;
9 import org.openqa.selenium.firefox.FirefoxDriver;
10
11 public class MainTest {
12     WebDriver driver;
13
14     ... Kode pengujian akan ditempatkan di sini ...
15
16     @AfterEach
17     void quit() {
18         if (driver != null) driver.quit();
19     }
20 }

```

A.2 Implementasi method validTest

Method validTest digunakan untuk menguji login dengan user "standard_user". Dalam method ini, mahasiswa membuka browser, mengakses situs saucedemo.com, memasukkan username dan password yang benar, kemudian menekan tombol login. Setelah itu, mahasiswa memverifikasi bahwa login berhasil dengan memeriksa apakah elemen tertentu yang hanya muncul setelah login berhasil dapat ditemukan. Berikut adalah implementasi method validTest:

```

1 @Test
2     void validTest(){
3         driver = new FirefoxDriver();
4         driver.get("https://www.saucedemo.com/");
5         WebElement userName = driver.findElement(By.id("user-
↪ name"));
6         userName.sendKeys("standard_user");
7         WebElement password = driver.findElement(By.id("
↪ password"));
8         password.sendKeys("secret_sauce");
9         WebElement submit = driver.findElement(By.id("login-
↪ button"));
10        submit.submit();
11
12        String expectedUrl = "https://www.saucedemo.com/
↪ inventory.html";
13        Assertions.assertEquals(expectedUrl, driver.
↪ getCurrentUrl(), "URL should match inventory page");
14
15        WebElement header = driver.findElement(By.className("
↪ title"));
16        Assertions.assertEquals("Products", header.getText());
17

```

```

18         boolean isCartVisible = driver.findElement(By.className
    ↪ ("shopping_cart_link")).isDisplayed();
19         Assertions.assertTrue(isCartVisible);
20     }

```

variabel driver digunakan untuk mengontrol browser, dan method get digunakan untuk membuka situs web. Kemudian, mahasiswa menggunakan findElement untuk menemukan elemen input username dan password, mengisi keduanya dengan sendKeys, dan menekan tombol login dengan submit. Setelah itu, mahasiswa memverifikasi bahwa URL saat ini adalah URL yang diharapkan setelah login berhasil, memeriksa apakah header "Products" muncul, dan memastikan bahwa ikon keranjang belanja terlihat.

A.3 Implementasi method lockedOutUser

Method lockedOutUser digunakan untuk menguji login dengan user "locked_out_user". Dalam method ini, mahasiswa melakukan langkah yang sama seperti pada validTest, tetapi dengan username yang berbeda. Setelah mencoba login, mahasiswa memverifikasi bahwa login gagal dengan memeriksa apakah pesan error yang sesuai muncul. Berikut adalah implementasi method lockedOutUser:

```

1     @Test
2     void lockedOutuser(){
3         driver = new FirefoxDriver();
4         driver.get("https://www.saucedemo.com/");
5         WebElement userName = driver.findElement(By.id("user-
    ↪ name"));
6         userName.sendKeys("locked_out_user");
7         WebElement password = driver.findElement(By.id("
    ↪ password"));
8         password.sendKeys("secret_sauce");
9         WebElement submit = driver.findElement(By.id("login-
    ↪ button"));
10        submit.submit();
11
12        WebElement errorElement = driver.findElement(By.
    ↪ cssSelector("[data-test='error']"));
13        String errorMessage = errorElement.getText();
14
15        Assertions.assertTrue(errorMessage.contains("Sorry,
    ↪ this user has been locked out"));
16    }

```

variabel driver, method get, findElement, sendKeys, dan submit digunakan dengan cara yang sama seperti pada validTest. Setelah mencoba login, mahasiswa mencari elemen yang menampilkan pesan error menggunakan findElement dengan selector CSS. Kemudian, mahasiswa memeriksa apakah pesan error yang ditampilkan mengandung teks "Sorry, this user has been locked out" untuk memastikan bahwa login gagal sesuai dengan yang diharapkan.

B. Gambar

Berikut adalah gambar hasil running pengujian validTest dan lockedOutUser:

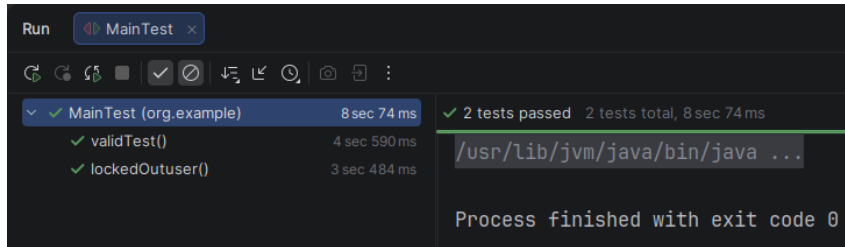


Figure 1: Hasil Running validTest